

Codex Web 工作台 v0.3：多通道网页 Agent、持久旁白与故障隔离

3099404236^{1*}

¹Independent developer

本报告记录 Codex Web 工作台的第三个公开版本。系统保留 Codex CLI 的文件、终端、补丁、Skills 与 MCP 工具循环，同时把主模型请求临时转发到用户已经登录的 ChatGPT 网页。v0.3 将此前的单页或双页控制扩展为三条工作通道与一条专用旁白通道。每条工作通道拥有独立的 Chrome DevTools MCP 进程、固定标签页、会话亲和关系和租约代数，从而允许不同任务并行，并阻止它们争抢同一页面。旁白通道使用独立持久对话，把已完成或失败的工作回合解释为可读进度；通道离线时事件本地排队，恢复后自动补放。启动器进一步加入多次直连探测、健康 3+1 拓扑防降级，以及仅匹配 Chrome 远程调试许可窗口的精确自动允许脚本。2026-08-21 的公开代码验证包含 91 项自动测试，全部通过。项目仍是实验性本机工具，不是 OpenAI 官方 API，也不绕过登录、验证码、风控或用量限制。

Codex | Responses API | ChatGPT Web | browser concurrency | observer agent | fault isolation

<https://github.com/3099404236/chatgpt-web-codex-bridge>

问题背景与版本演进

初代版本证明了一条最小链路可以成立：Codex 继续拥有工具，网页模型只返回一个工具调用或最终回答，桥接器把网页输出转换为 Responses JSON/SSE。第二版解决了跨重启 URL、同页恢复、请求去重和结构化检查点。第三版面对的是另一个更接近真实使用的问题：当多个 Codex 任务、Research 子任务和旁路解释器同时存在时，怎样避免它们互相刷新标签页、占满通道，又怎样让用户看懂系统正在做什么？

版本	定位	主要能力
v0.1	最小桥接	Responses JSON/SSE、同对话工具续轮、shell 与 apply_patch 协议、30 项测试。
v0.2	可恢复工作台	持久 URL、同页故障恢复、检查点、请求去重、长文本分块、工作流面板、52 项测试。
v0.3	并发与可解释观测	三条粘性工作通道、一条持久旁白、差异化租期、离线补放、拓扑防降级、自动许可、91 项测试。

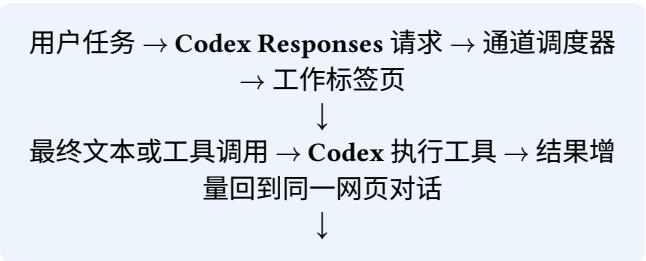
三个公开版本的能力边界。个人主页保留每一版 PDF，v0.3 不覆盖历史文件。

本项目的目标不是把 ChatGPT 订阅转换成稳定 API 额度，而是在用户明确登录且理解调试权限的前提下，研究黑盒网页如何参与本机 Agent 循环。网页 DOM、网络和模型格式可能变化，因此系统更重视身份、终态、幂等、隔离和可诊断性，而不是伪装成生产级接口。

v0.3 系统架构

组件	端口或数量	职责
Codex TUI	-	组织 Agent 循环，声明工具，执行文件、终端和补丁操作。
Responses 桥	8766	协议转换、会话状态、通道调度、请求去重、检查点和明确终态。
工作通道	3	每条通道包含独立 MCP 进程与固定 ChatGPT 标签页，服务不同任务。
旁白通道	1	固定持久对话，只解释工作回合；不执行工具，不进入主调度池。
旁白页面	8766/observer	显示正在解释、等待通道、已解释和失败，并通过 SSE 实时刷新。
Supervisor	8770	保存工作区、任务、人工阶段和 App Server 事件，提供主工作台。
App Server 代理	8772	透明转发 Codex App Server，并把有界事件副本送入工作台。
Browser Relay	9224	直连路径不可用时的本机 fallback；不复制 Cookie 或 profile。

v0.3 的本机组件。自建网络服务默认绑定 127.0.0.1。



脱敏回合事件 → 旁白队列 → 独立旁白对话与本地
时间线

主链路与旁白链路只在“回合事件”处单向相交。旁白慢、离线或格式错误时，主 Responses 请求仍可完成；主链路不会等待旁白回答，也不会把旁白建议当作工具指令。

从共享页面到固定 worker

早期并发尝试把多个任务挂在同一个浏览器控制实例上。Chrome DevTools MCP 的页面选择是 进程内状态；当两个调用交错执行 select_page、导航和快照时，后一次选择可能改变 前一次操作的目标。表面现象是两个终端不断刷新第一个标签页，第二个标签页保持空闲，或某个任务在错误对话中完成。多个独立 MCP 进程连接同一调试端点也可能放大握手、许可和协议超时，相关现象在上游 issue 中已有报告¹。

v0.3 使用固定 worker 模型：每条工作通道在启动时创建自己的 MCP 进程并固定一个页面偏移，每轮操作前重新确认页面身份。通道池只把任务交给 worker，不共享可变的 selected page。调度器同时维护四条不变量：

- 同一任务严格串行，即使其他通道空闲也不并发发送同一会话；
- 不同任务可以占用不同 worker 并行执行；
- 每次租约包含递增代数，上一代遗留的异步操作会被拒绝；
- 任务有排队工作时不释放亲和关系，完成后再按任务类型决定租期。

请求类型	空闲租期	理由
VS Code 交互任务	20 分钟	用户可能短暂停顿后继续修改；保留标签页可以减少上下文切换和错误导航。
Research 搜索或证据批次	队列清空即释放	子任务通常密集出现后长期空闲，立即释放避免五个报告分片占满全部通道。
同一任务仍有排队项	不释放	防止后续项被其他任务插入并打乱会话顺序。
持久会话 URL	永久保留	释放运行通道不等于删除任务身份；之后 /resume 仍可回到旧对话。

运行资源租期与持久会话身份必须分开建模。

排队优先级为交互式 Codex、搜索任务、长报告或证据批次。这个优先级只决定谁先获得空闲 worker，不改变任务内部顺序，也不允许抢占正在运行的网页生成。

会话 URL、侧边栏与跨重启恢复

Codex 自己的 /resume 恢复本地任务历史，桥接器则恢复这个任务绑定的 ChatGPT 对话 URL。两者属于不同层：只恢复 Codex 而不恢复 URL，会让同一任务误入新网页对话；只保存网页 URL 而不保存同步位置，又可能重放完整历史。

恢复过程：任务键命中本地状态，worker 打开或选择固定标签页，优先扫描侧边栏中的真实 对话链接并点击精确 URL，侧边栏确实找不到时才使用直接导航。页面到达后再次核对 route，只发送尚未同步的最近增量。Chrome 重启后页面编号可能重新绑定到不同页面，上游也报告过 旧 page id 静默指向新页面的风险²；因此 v0.3 不把 page id 当作永久身份，真正的持久身份仍是 conversation URL。

普通超时、断流、Thinking、网关错误、标签页关闭和 DevTools 断线都只记为暂态失败，旧 URL 继续保留。只有页面连续三次明确报告对话不存在、已删除、不可用或无权访问，才允许 预留替代对话。替代对话的压缩交接在本版仍保持“已预留但未自动启用”，避免在结果未知时 擅自新开对话并重放巨大上下文。

官方 Responses 可以使用 previous response id、Conversation 对象与 compaction 维持长任务 状态^{3,4}。网页桥无法获得这些服务器端对象，所以本地检查点只保存可审计的外显事实：原始目标、最近输入、工具结果、最后确认完成的输出、同步指纹和源 URL。它不是模型隐藏推理，也不尝试解析不透明 compaction item。

持久旁白：观察而不干预

用户需要知道 Agent 在做什么，但把所有本地事件原样显示会得到长命令、协议字段和难以理解 的工具输出。v0.3 新增专用旁白对话：主桥在每轮结束时构造脱敏事件，旁白模型将其翻译为 简短、通俗的进度说明，本地页面按时间顺序展示。

状态	页面含义	恢复规则
pending	事件已进入队列	同一进程内按顺序解释。
explaining	旁白网页正在生成	主链路继续运行，不等待该结果。
waiting_for_channel	旁白通道暂时离线	事件保存在本地；指数退避后重试，或下次启动自动补放。
explained	解释已持久保存	刷新页面或重启仍可查看。
failed	非暂态配置或内容错误	保留错误原因，避免静默丢失。

旁白状态机。暂态断线不会立即变成永久失败。

旁白通道固定一个 ChatGPT URL，不为每条事件新开对话。事件日志会对常见 token 和密钥形态做脱敏；公开仓库不包含真实旁白记录。旁白的角色是“翻译发生了什么”，不是“评价下一步该执行什么”，因此它不接收 shell、补丁或其他工具权限。

Chrome 连接、许可堆叠与启动拓扑

Chrome 从 136 起收紧默认数据目录的远程调试开关，官方建议自动化使用自定义 user data 目录或 Chrome for Testing⁵。本项目的实验目标恰恰是复用用户已登录的日常网页，因此采用 chrome://inspect/#remote-debugging 的显式许可路径，并保留本机 Browser Relay 作为 fallback，而不是绕过用户授权。

四个独立 MCP 客户端会产生多次许可请求。上游 issue 1794 描述了相同提示堆叠、需要连续确认的现象⁶。v0.3 的启动器在连接窗口内启动一个短期 Windows UI Automation 监听器。它不使用坐标、全局按键或模糊文字，只接受以下同时成立的条件：顶层窗口属于 chrome.exe；窗口中存在精确的远程调试提示；目标控件类型为 Button；按钮名称精确等于 Allow 或“允许”。全部提示消失并持续空闲 30 秒后监听器退出。用户可用 CODEX_WEB_AUTO_ALLOW_REMOTE_DEBUGGING=0 完全关闭该功能。

启动探测也不能只做一次。一次 WebSocket 握手超时可能来自许可尚未处理、Chrome 短暂卡顿或 DevTools ActivePort 状态切换。v0.3 最多进行五次退避探测并记录每次原因。更重要的是，如果现有桥已经达到三条工作通道可用且旁白可用，一次新预检“不确定”不会停止旧进程并静默降级成单通道。拓扑变更必须以实际健康状态为依据。

网页输入、完成检测与剩余脆弱点

网页没有本地可订阅的官方 response completed 事件。桥接器联合观察停止按钮、输入框、发送或语音控件、最新 assistant 文本稳定性、结构化工具块闭合和整页网关错误。可见的 Thinking、Working 或 Analyzing 永远不是最终回答。流式请求最终必须显式产生 response.completed 或 response.failed。

长提示按块写入编辑器，并校验页面内缓冲区长度。写入完成后，桥接器用真实键盘事件唤醒网页框架的内部状态，再等待发送按钮。2026-08-20 的现场日志还暴露了一个未完全解决的脆弱点：DOM 中出现文字后，ChatGPT 前端偶尔仍把编辑器判定为空，发送按钮保持禁用，最终报错“did not enable its send button”。系统选择失败而不是盲按回车，避免把空消息或错误页面送出；后续应加入清空、重新聚焦、全量可信键盘输入和同 URL 刷新重试。

安全优先的失败原则：当页面是否真正接收输入、是否已经发送或是否仍在生成不能确定时，返回明确失败比自动重复点击更安全。未知结果重发可能造成重复消息和重复工具执行。

从踩坑到系统规则

现场现象	根因或风险	v0.3 规则
两个终端互相刷新第一个标签页	共享 MCP 的页面选择状态被交错修改	每条 worker 独立 MCP、固定标签页、每轮重选并核对 URL。
长报告五个分片占满通道	空闲 Research 亲和关系仍按交互任务租期保留	Research 队列清空即释放，交互任务保留 20 分钟。
第九个会话挤不进去	把历史会话数误当作正在运行的槽位	持久 URL 表与运行 worker 池分离；只有忙碌和排队占资源。
启动偶尔只剩一个通道	一次直连预检失败触发全局降级	五次探测；已有健康 3+1 时保持原拓扑。
每次启动要点多次允许	独立 CDP 客户端产生堆叠许可	精确 UIA 监听，按提示消失和空闲时间退出。
旁白显示“没有通道”	观察通道与工作池竞争或启动失败	旁白专用 lane；离线事件进入 waiting 状态并补放。
Thinking 被当作最终回答	只看见新文本，没有确认页面空闲	占位符过滤、控件状态、文本稳定与结构闭合联合判定。
发送按钮始终禁用	DOM 文本与前端内部状态不同步	不发送；记录暂态失败，保留原 URL，预留可信输入重试。

v0.3 把现场问题转换为可测试的调度、身份与恢复规则。

实现与验证

公开代码保留网页桥接核心，不包含手机通知服务、QQ 机器人、VPS 配置或任何凭据。新增 主要模块为 browser-lane-pool.mjs、observer-service.mjs、allow-chrome-remote-debugging.ps1 和 probe-chrome-ws.mjs；server.mjs、chrome-mcp-browser.mjs 与启动器也同步更新。

2026-08-21 在独立公开仓库执行 npm ci、npm run check 和 npm test。依赖审计报告 0 个已知漏洞；语法检查通过；91 项测试全部通过，0 项失败。

测试组	数量	主要覆盖
浏览器通道池	13	并发、粘性会话、差异化租期、排队、取消、租约代数与优先级。
Chrome MCP 控制	17	页面固定、URL 恢复、侧边栏、长输入、错误页、断线重连与完成判定。
旁白服务	4	持久对话、脱敏、暂态重试、离线落盘补放和独立页面。
Responses 服务	31	JSON/SSE、工具循环、会话恢复、检查点、去重、研究租期与明确终态。
工具协议与补丁	15	YAML/JSON、Windows 路径、原生补丁、边界检查和错误识别。
工作台与通知接口	11	SQLite 状态、人工阶段、App Server 事件和可选注意力通知。
合计	91	全部通过，0 失败。

v0.3 公开代码的自动化测试分组。

真实账号相关验证不进入自动测试。2026-08-20 的本机冷启动检查显示三条工作通道全部 ready，旁白通道 ready，队列为 0；自动许可监听器在提示消失后退出。这个结果证明当次环境可用，不代表网页 DOM 或 Chrome 授权机制长期不变。

公开发布与安全边界

公开仓库排除 ChatGPT Cookie、浏览器 profile、真实任务历史、对话 URL、日志、状态数据库、截图、API key、DPAPI 文件、本机私用 ZIP、手机与 VPS 配置。发布候选文件在复制前后都进行 凭据形态和绝对本机路径扫描。用于测试 Windows 反斜杠修复的真实目录样例也替换为虚构路径。

边界	当前措施	剩余风险
网络暴露	自建服务只监听 127.0.0.1	同机其他进程仍可能访问本地端口。
浏览器登录	不复制 Cookie 或 profile；需要用户显式许可	调试已登录标签页本身拥有很强权限。
自动允许	仅精确 Chrome 窗口、提示和 Button；短期运行，可关闭	若 Chrome 改变无障碍名称，功能会失效而不是自动适配。
工具执行	网页只返回白名单调用，实际由 Codex 执行	danger-full-access 下 Codex 权限很高，需要可信任务。
旁白	只读回合概要、脱敏、无工具权限	网页模型仍会看到被送去解释的概要内容。
网页稳定性	错误显式化、同 URL 恢复、请求去重	DOM、服务状态和产品条款可能变化。

现有措施降低误操作，但不构成浏览器或操作系统级安全隔离。

复现步骤

公开代码位于 3099404236/chatgpt-web-codex-bridge。在 Windows PowerShell 中：

```
cd code
npm install
./scripts/install-codex-web.ps1
codex-web setup chrome
codex-web doctor
```

然后在任意工作目录运行 codex-web。主工作台为 http://127.0.0.1:8770/，旁白页面为 http://127.0.0.1:8766/observer。关闭终端后可再次启动并在 Codex TUI 中使用 /resume 恢复旧任务及其持久网页 URL。

局限、后续工作与结论

v0.3 解决的是多任务争抢、旁路观测和启动拓扑，不是网页 API 的根本不稳定性。当前仍有四个主要局限：网页输入状态可能与 DOM 不同步；多个调试客户端的许可和协议开销较大；旁白解释依赖另一个网页模型；替代对话的压缩交接尚未自动启用。

后续工程工作应优先完成两项：第一，为发送按钮未启用加入可审计的清空、重新聚焦、可信键盘重输与同 URL 刷新重试；第二，为 worker 启动加入逐通道恢复，使单条连接失败可以局部补齐，而不是依赖整体重启。研究方向则是比较自然语言摘要、结构化检查点、官方不透明 compaction、开源模型 KV cache 和隐藏状态压缩在长工具任务中的恢复质量。

第三版的主要结论是：网页型 Agent 的“会话身份”“运行通道”“任务租期”“观察通道”和“启动探测”必须分别建模。会话可以永久保留而 worker 立即释放，旁白可以暂时离线而主链路继续，预检可以不确定而健康拓扑不降级。只有拆开这些概念，并为每一层设置明确终态和恢复规则，多网页并发才不会退化成互相刷新、静默丢事件和无休止的人工作业。

参考文献

1. ChromeDevTools chrome-devtools-mcp contributors. Multiple MCP processes on the same debug port cause Network enable timeout. <https://github.com/ChromeDevTools/chrome-devtools-mcp/issues/1763> (2026).
2. ChromeDevTools chrome-devtools-mcp contributors. Page ids silently rebind to different pages after a browser restart. <https://github.com/ChromeDevTools/chrome-devtools-mcp/issues/2339> (2026).
3. OpenAI. Conversation State. <https://developers.openai.com/api/docs/guides/conversation-state> (2026).
4. OpenAI. Compaction. <https://developers.openai.com/api/docs/guides/compaction> (2026).
5. Chrome for Developers. Changes to remote debugging switches to improve security. <https://developer.chrome.com/blog/remote-debugging-port> (2025).
6. ChromeDevTools chrome-devtools-mcp contributors. New chrome inspect Allow remote debugging popup can trigger multiple times and stack up. <https://github.com/ChromeDevTools/chrome-devtools-mcp/issues/1794> (2026).